

DEFINING THE MISSION



1. UPLOAD

Send files to remote storage from any device.



2. SYNC & DOWNLOAD

Automatic propagation of changes across devices. Retrieve files anywhere.



OUT OF SCOPE:

Real-time collaborative editing (e.g., Google Docs).

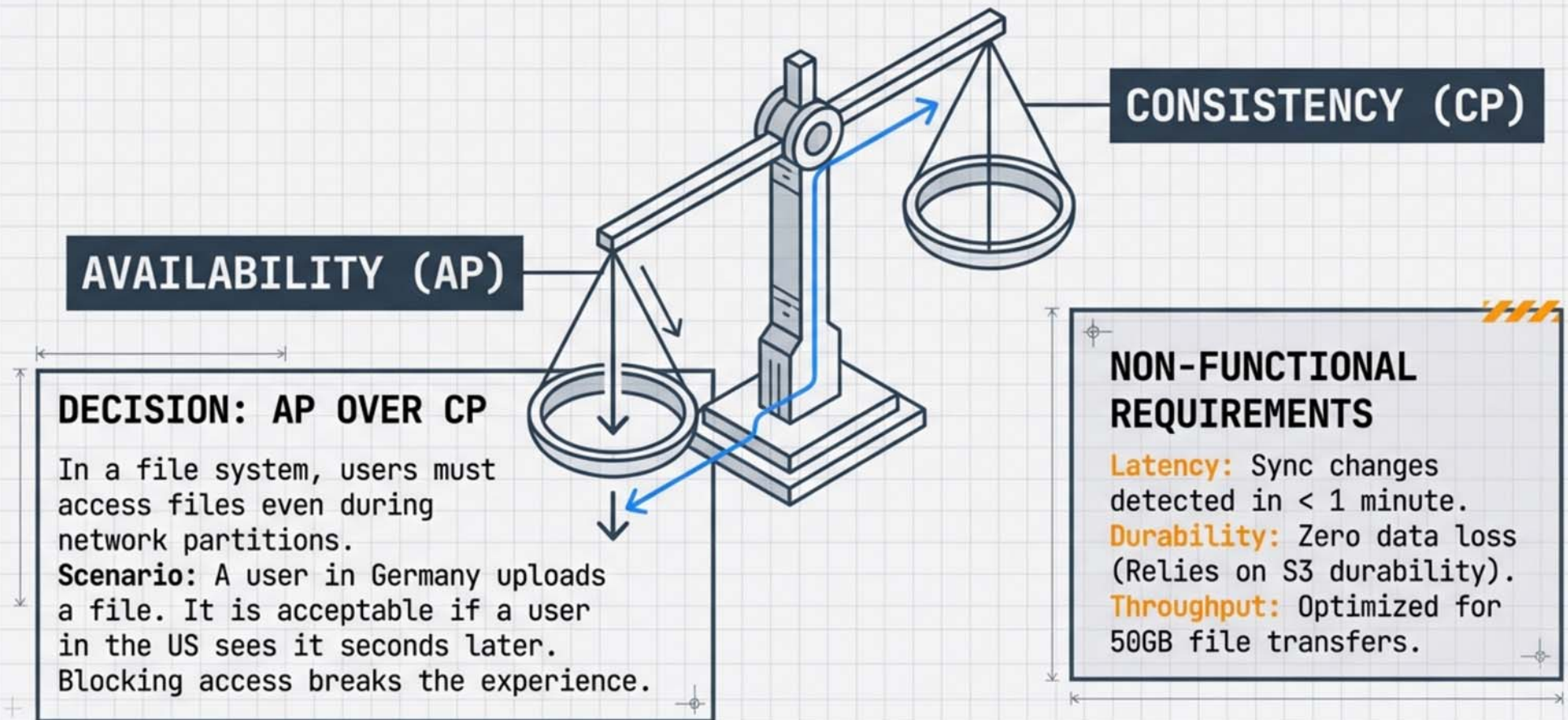


OUT OF SCOPE:

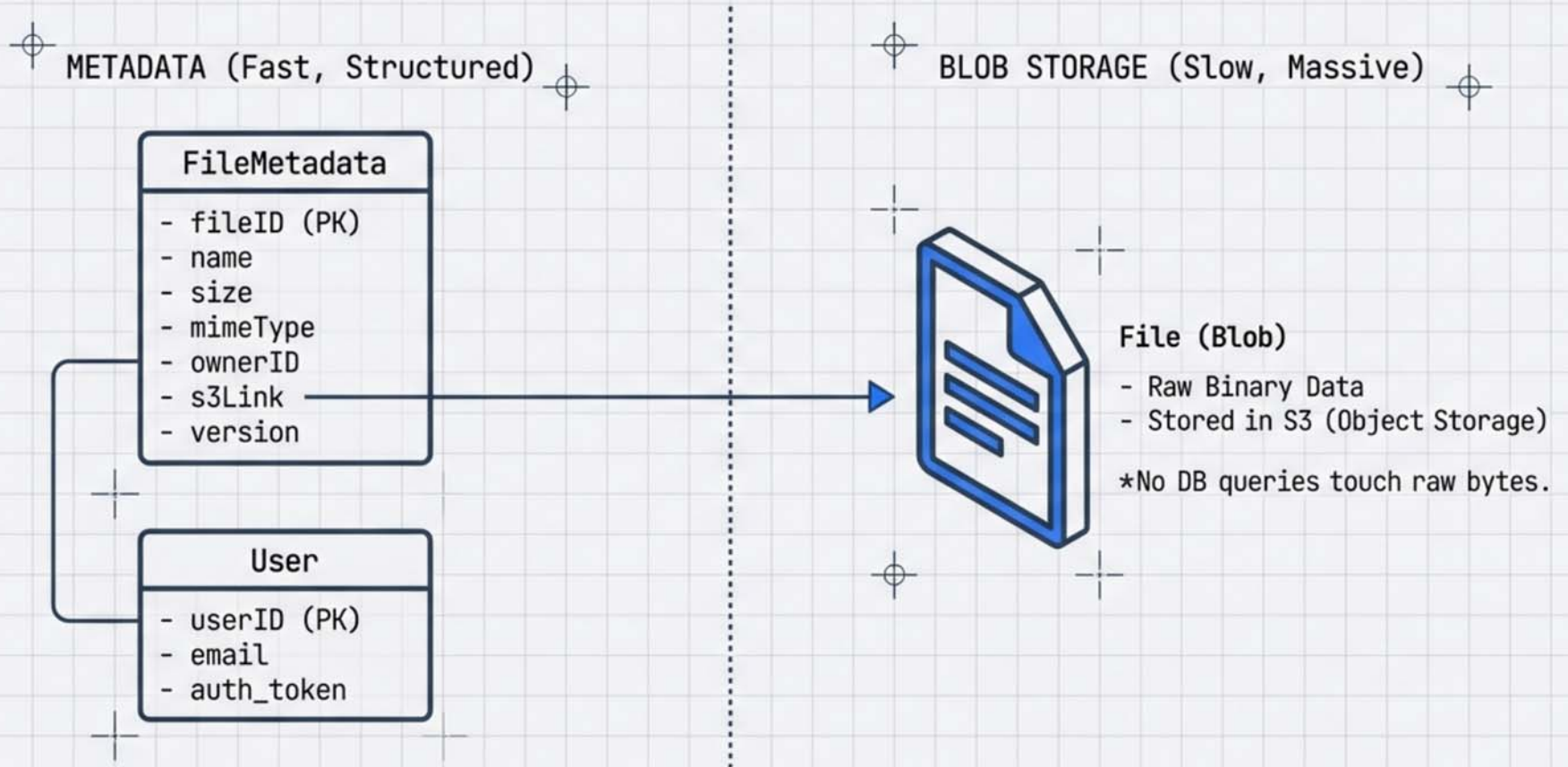
Designing the Blob Storage Engine (e.g., building S3 from scratch).

KEY INSIGHT: The challenge isn't just storage; it's keeping the local file system in perfect harmony with the cloud.

THE RULES OF THE ROAD: CAP THEOREM

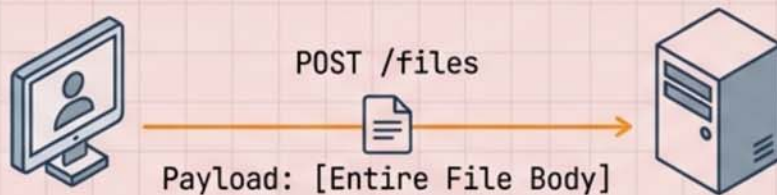


CORE ENTITIES: DATA MODEL



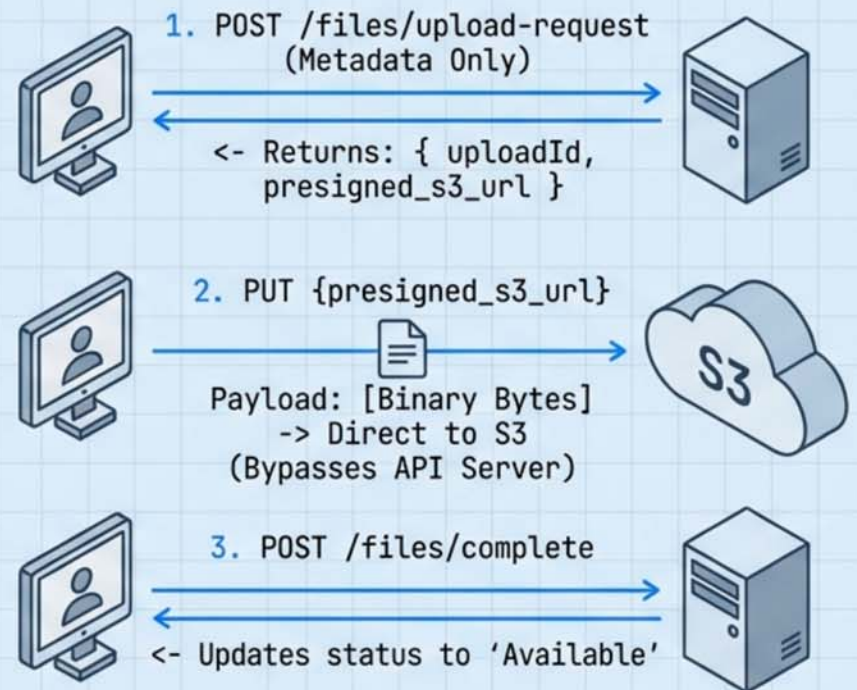
THE SYSTEM INTERFACE (API EVOLUTION)

THE NAIVE APPROACH (FAILS AT SCALE)

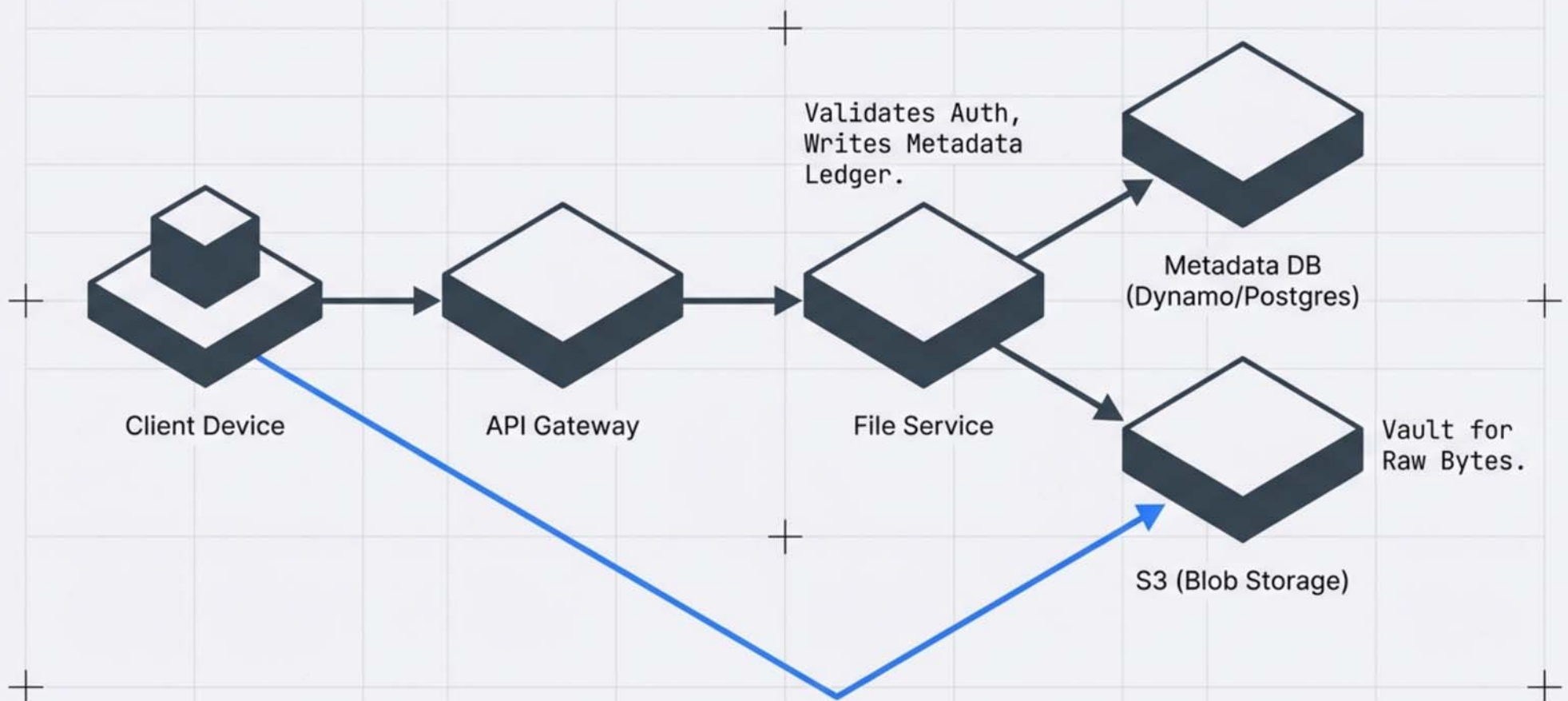


- Gateway Timeouts (10MB limits)
- Server Memory Exhaustion
- Connection Drops fail entire upload

THE SCALABLE APPROACH (PRESIGNED URLs)

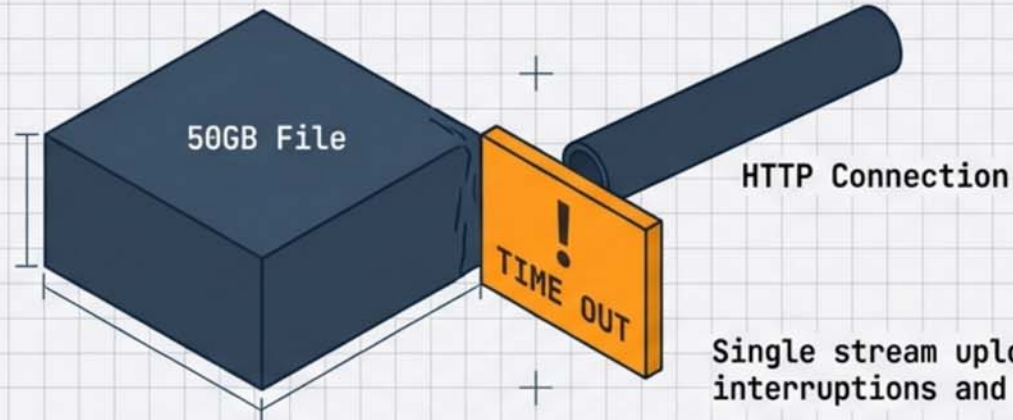


HIGH-LEVEL ARCHITECTURE: THE HAPPY PATH



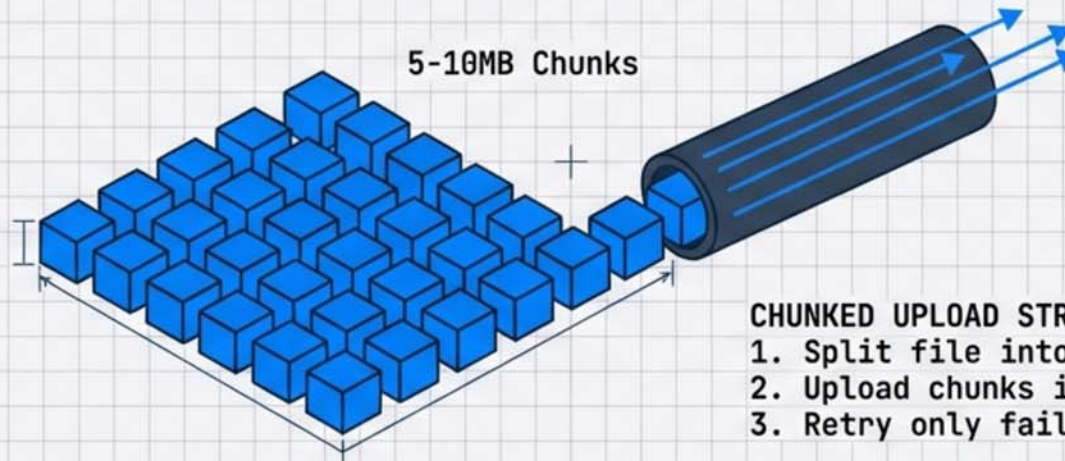
THE 50GB PROBLEM: SOLVING FOR SCALE

THE PROBLEM



Single stream uploads fail due to network interruptions and gateway limits.

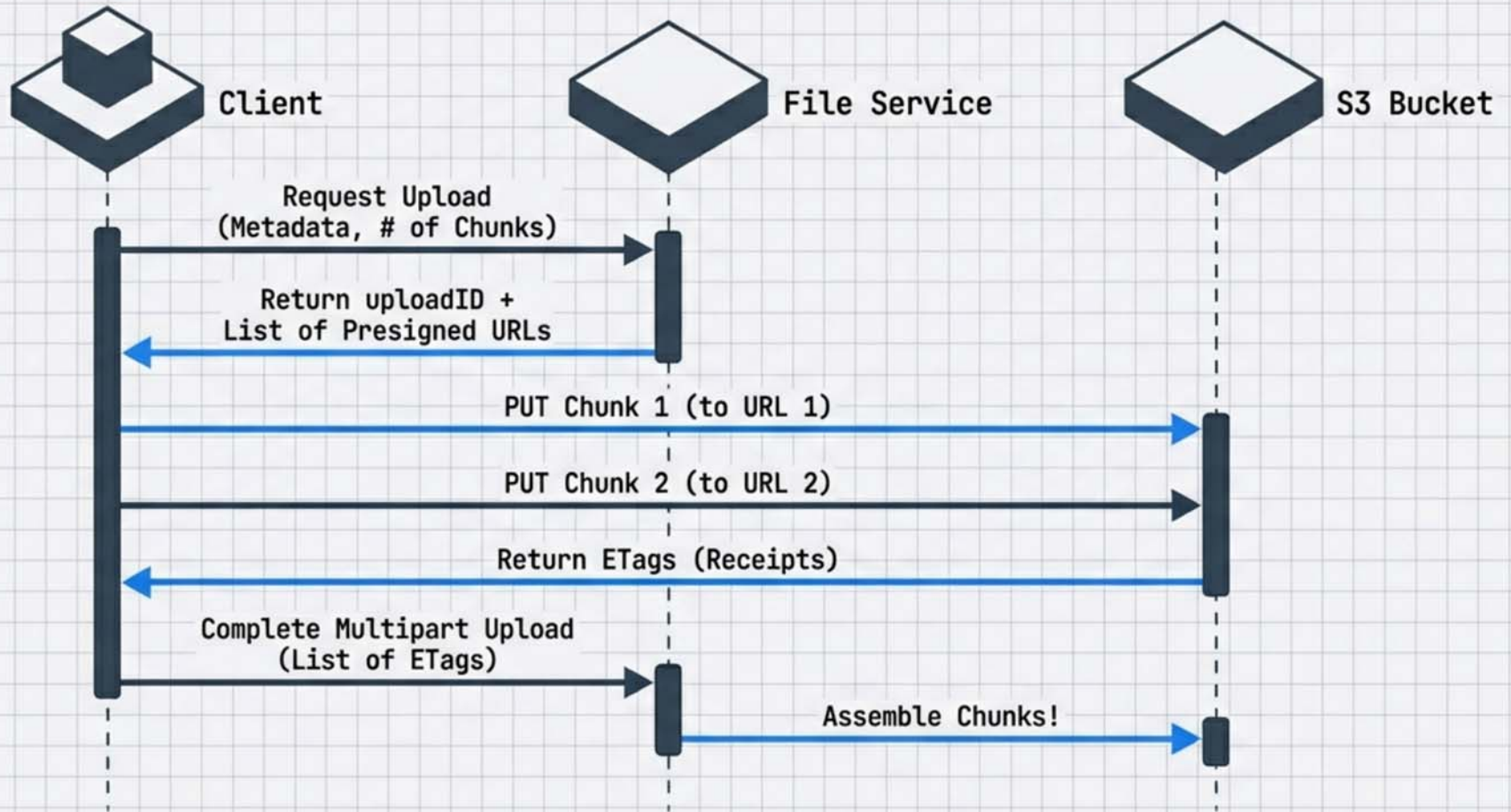
THE SOLUTION



CHUNKED UPLOAD STRATEGY

1. Split file into 5-10MB chunks on Client.
2. Upload chunks in parallel.
3. Retry only failed chunks.

OPTIMIZING UPLOAD: THE PRESIGNED URL HANDSHAKE

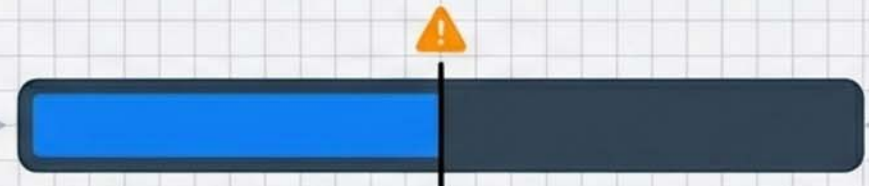


INTEGRITY & RESUMABLE UPLOADS



FINGERPRINTING (HASHING)

- Calculate SHA-256 hash for the whole file.
- Calculate SHA-256 hash for each chunk.
- The hash acts as the content addressable ID.

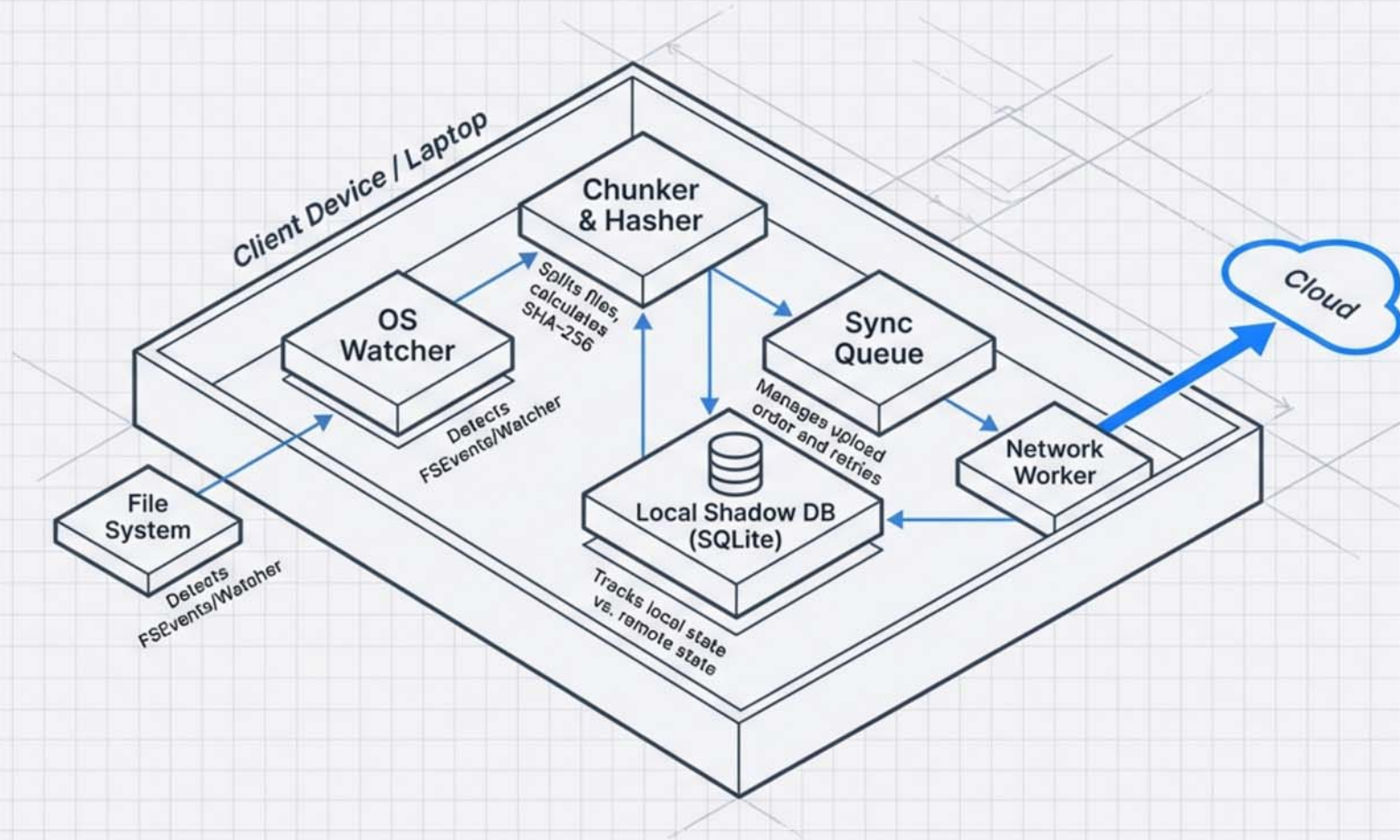


PAUSED AT 50%

RESUME LOGIC: TRUST BUT VERIFY

1. Connection Drops.
2. Client asks DB: "What chunks do you have for File Hash X?"
3. Server verifies presence in S3.
4. Server responds: "I have chunks 1-50. Send 51-100."
5. Upload resumes without data duplication.

THE INTELLIGENT CLIENT: SYNC AGENT ANATOMY



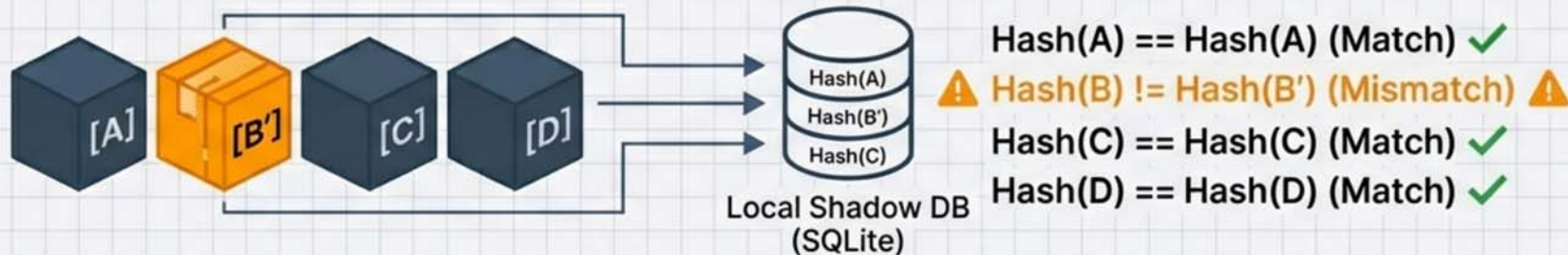
DELTA SYNC: SPEEDING UP THE FLOW

1. User edits the file.

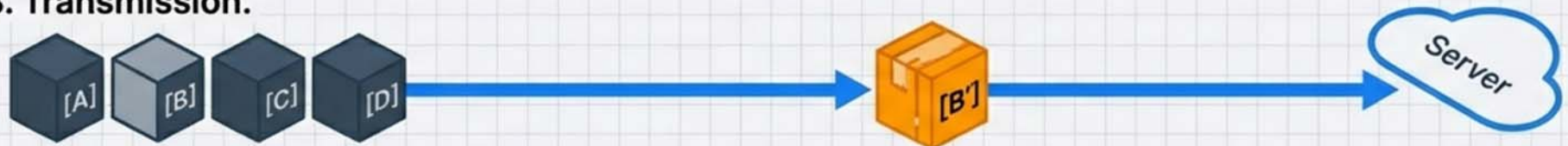


2. Hashing & Comparison.

Client compares new hashes to Local Shadow DB.

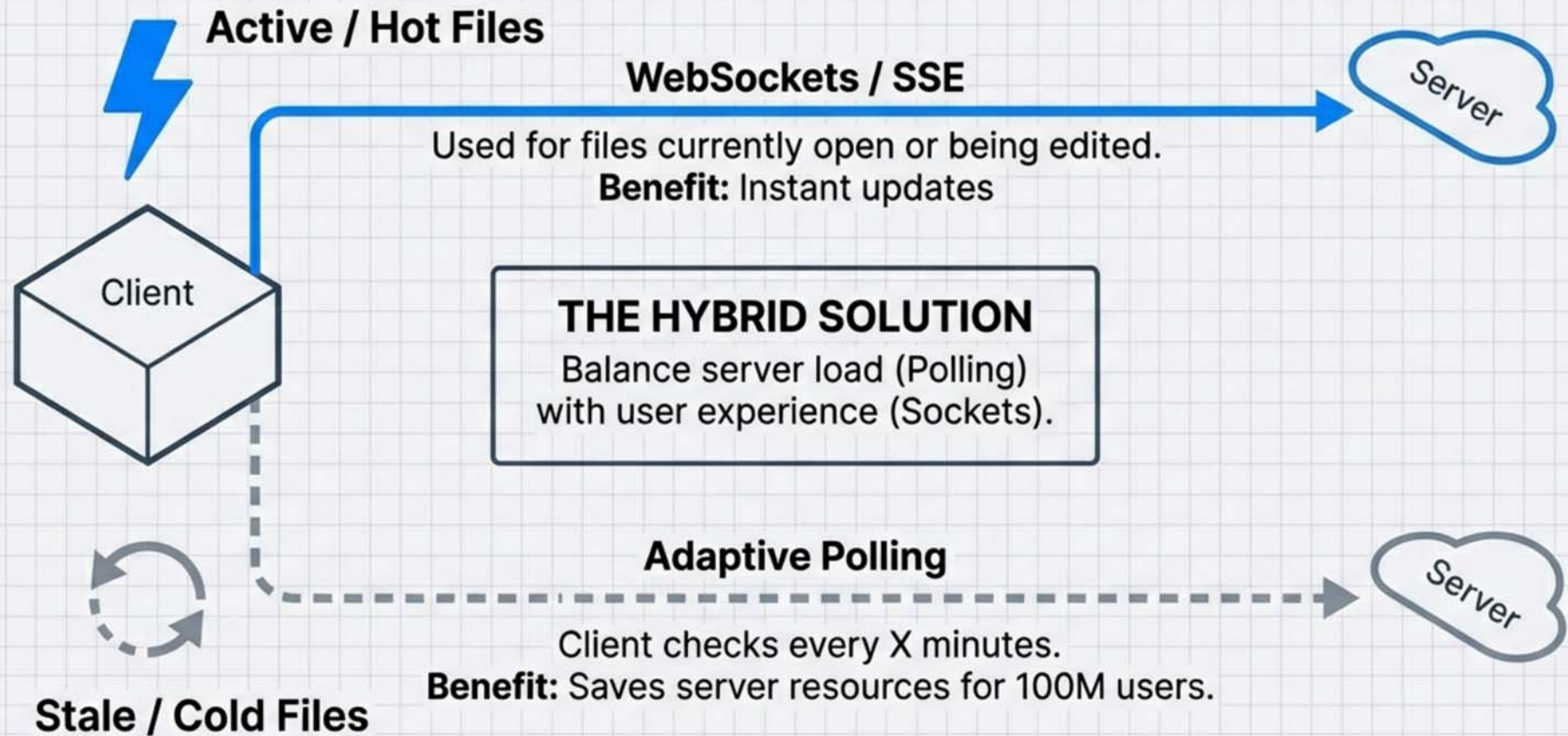


3. Transmission.

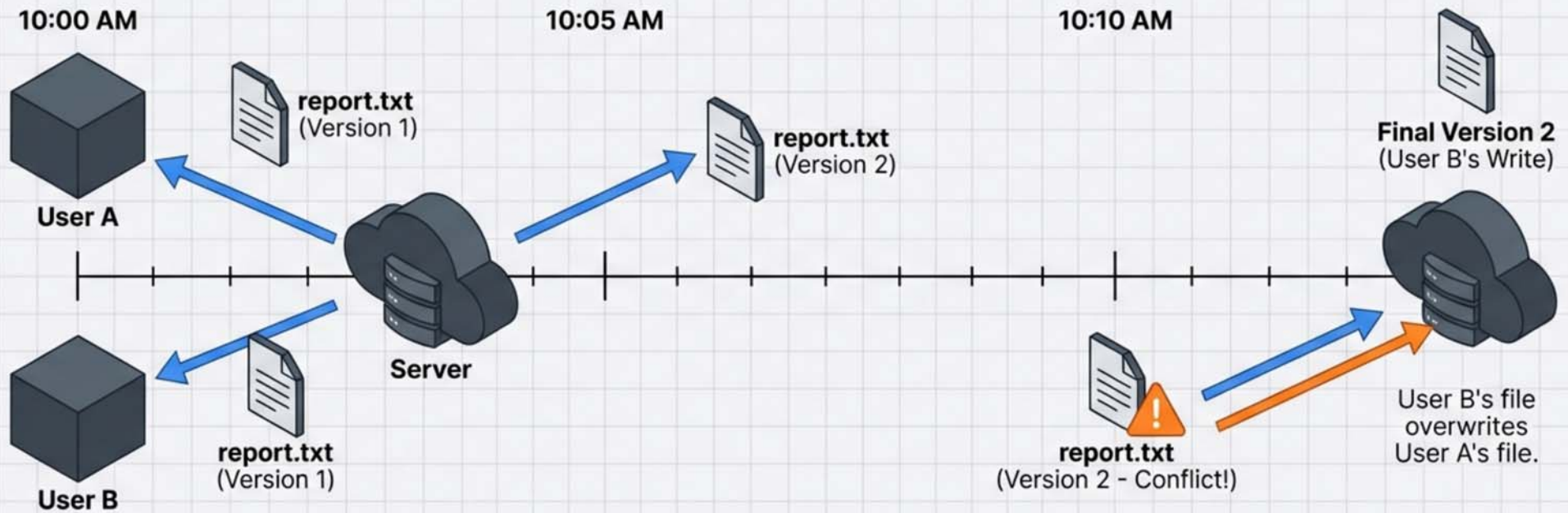


Result: Only modified bytes are transmitted. **Massive bandwidth reduction.**

REMOTE-TO-LOCAL: POLLING VS. PUSH



HANDLING CONFLICTS: LAST WRITE WINS



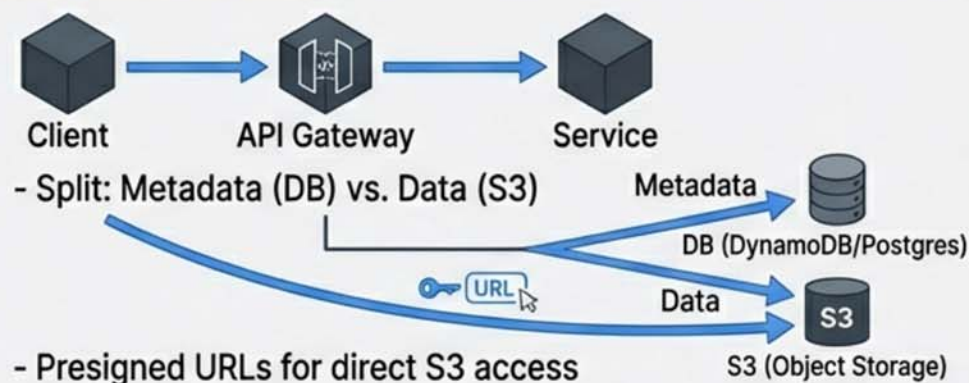
Strategy: Last Write Wins (LWW) based on server timestamp.

Trade-off: Simple but risks overwriting concurrent work.

Advanced Alternative: Keep conflicting copies (e.g., `'report (conflicted copy).txt'`).

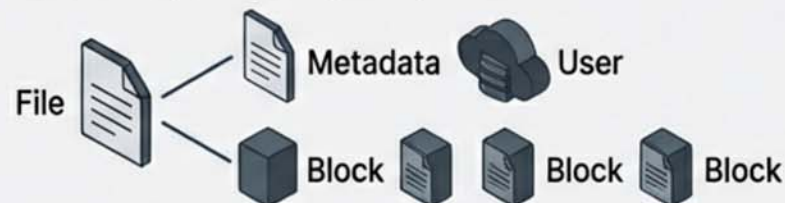
SYSTEM DESIGN SUMMARY

ARCHITECTURE



DATA MODEL

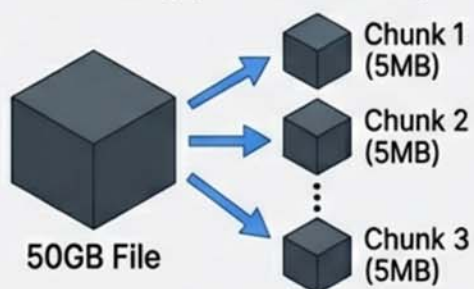
- Entities: File, Metadata, User, Block



- Metadata: DynamoDB/Postgres
- Storage: S3 (Object Storage)

SCALING (50GB)

- Chunking (5-10MB blocks)



- Multipart Upload (Parallel)



- Resume on failure (Check hashes)



SYNC LOGIC

- OS Watcher (FSEvents)



- Delta Sync (Transmit changes only)

- Hybrid Polling/Sockets

